UNIWERSYTET VIZJA

FIELD OF STUDY: COMPUTER SCIENCE

Team Project

Group –1

Chinkhusel Tsolmonbaatar

Aghamir Ahmadov

Anastasiia Chaika

Nadege Uwase

Samreen Begum

Professor : **Marcin Kacprowicz**

Identity Access Management System

Problem Statement:

Organizations often struggle to manage who has access to which systems, especially when employees join, change roles, or leave. For example, an employee leaving a company may still have access to sensitive internal tools like Slack, payroll systems, or code repositories if IT doesn't revoke permissions promptly. A centralized Identity and Access Management system ensures proper authentication, authorization, and auditing across all internal systems, reducing security risks and administrative overhead.

The **goal** of this project is to develop an Identity and Access Management (**IAM**) system that provides secure authentication and efficient control of user roles and permissions. The system will help administrators manage users while ensuring secure access to resources

Target Users:

1. **System administrators** for granting, modifying, and revoking user access.
2. **Companies** and **Organizations** or development teams that need a secure authentication system
3. **End users** who need safe and reliable login to applications

Main Features:

1. **User Authentication:** Secure login, registration, and password recovery.
2. **Role-Based Access Control (RBAC):** Assign roles (Admin, Manager, Employee) to control access to different resources.
3. **Token-Based Security:** Use JWT tokens for secure API access.
4. **Audit Logging**: Track user actions to ensure compliance with standards like GDPR or HIPAA.
5. **API Integration:** Allow internal systems (e.g., payroll, project management tools, internal databases) to securely connect to the IAM system.
6. **User Management**: Programmatic interface for adding, modifying, or revoking users.

Technology Plan:

1. **Backend/API:** Node.js with Express or Python with FastAPI for authentication and authorization services.
2. **Database:** PostgreSQL or MongoDB for storing users, roles, and audit logs.
3. **Security**: JWT for session management and bcrypt for password hashing.
4. **Documentation**: API documentation in /docs folder using Markdown.

Problem Analysis

➢ Context and Motivation:

In modern organizations, employees rely on many different digital tools to perform their daily work. These tools can include communication platforms such as Slack or Microsoft Teams, internal databases, project management tools, payroll systems, and development platforms like Git repositories. Each of these systems typically requires users to have accounts and permissions to access certain resources.

Managing access to all these systems can quickly become complicated, especially as organizations grow. New employees join the company, existing employees change roles, and others eventually leave the organization. Every time one of these changes occurs, someone must update the user's permissions across multiple systems.

For example, when a new employee joins a company, the IT administrator may need to create accounts in several different tools such as the company's email system, communication platforms, and internal software. If this process is handled manually, it can take time and may lead to mistakes. In some cases, a system might be forgotten, leaving the employee unable to access the tools they need to do their job.

Another serious problem occurs when employees leave the organization, but their access is not revoked immediately. Imagine a developer who still has access to a company code repository after leaving the company, or a former employee who can still log in to internal systems containing sensitive company data. Situations like this create potential security risks.

An Identity and Access Management (IAM) system helps solve this problem by providing a centralized way to manage authentication and user permissions. Instead of managing access separately for each application, administrators can manage users, roles, and permissions from one system. This makes the process more secure, efficient, and easier to maintain.

Our project aims to demonstrate how an IAM system can help organizations control access to their systems while reducing the risk of security breaches and administrative errors.

➢ User Roles:

The proposed system will involve several types of users who interact with it in different ways.

System Administrator are responsible for managing the IAM system. Their tasks include creating user accounts, assigning roles, modifying permissions, and removing access when employees leave the organization.

- **For example**, if a new employee joins the development team, the administrator can assign them a "Developer" role that automatically allows access to development-related resources while restricting access to financial systems or administrative tools.

Administrators are also responsible for monitoring system activity and ensuring that access permissions are updated whenever organizational changes occur.

End Users are employees or individuals who need access to applications within the organization. They use the system primarily for authentication, meaning they log in to access different services.

- **For example**, an employee may log in to a company's application using their credentials, and the IAM system will verify their identity before allowing access. The level of access they receive depends on their assigned role within the organization.

Organizations and Development Teams can integrate the IAM system with their internal applications through APIs. This allows applications to verify user identity and permissions before granting access.

- **For instance**, when a user attempts to access an internal payroll system, the application can request verification from the IAM system to confirm whether the user has permission to view payroll information.

➢ Main Pain Points:

Many organizations face several challenges when managing user access across multiple systems.

One major issue is manual user management. When administrators must manage accounts across different platforms individually, the process becomes time-consuming and prone to errors. Creating or updating accounts for each system separately increases the chances that something will be missed.

Another important challenge is security risks caused by outdated permissions. If access rights are not updated when employees change roles or leave the organization, unauthorized users may still have access to sensitive data. For example, if a former employee still has access to internal company systems, they may unintentionally or intentionally access confidential information.

Organizations also face difficulties with monitoring and accountability. Without proper logging and monitoring, it can be difficult to determine who accessed certain resources or when a specific action occurred. This lack of visibility can make it harder to investigate security incidents or comply with data protection regulations.

Additionally, many systems implement their own authentication mechanisms, which leads to inconsistent access control policies. Managing security across different systems becomes complicated when each application handles authentication differently.

These challenges demonstrate the importance of implementing a centralized IAM system that can securely manage user identities, permissions, and activity logs.

➢ Success Criteria

To evaluate the effectiveness of the proposed **IAM** system, several success criteria can be considered.

First, the system should provide secure authentication for users. Users must be able to log in safely, and their passwords should be stored securely using **encryption techniques** such as **bcrypt** to prevent unauthorized access if the database is compromised.

Second, the system should correctly implement Role-Based Access Control (**RBAC**). Users should only be able to access resources that match their assigned roles. For example, a regular employee should not have access to administrative settings or confidential financial data.

Another key measure of success is efficient user management for administrators. Administrators should be able to easily create users, assign roles, update permissions, and revoke access through the system without needing to modify multiple applications individually.

The system should also provide audit logging, which records important actions such as login attempts, permission changes, and system access. These logs help organizations track activity and improve security monitoring.

Finally, the **IAM** system should support **API** integration with other applications, allowing different internal systems to rely on the **IAM** service for authentication and authorization.

If these criteria are met, the system will successfully demonstrate how centralized identity and access management can improve security, simplify user management, and ensure proper control over access to organizational resources.

Scope Definition

➢ In-Scope Features (Must-Have)

The following features are included in the scope of this project and will be implemented as part of the final system demonstration.

User Authentication
The system will provide secure user authentication, allowing users to register, log in, and verify their identity before accessing protected resources. Passwords will be securely stored using hashing techniques such as bcrypt.

Role-Based Access Control (RBAC)
The system will implement role-based access control to manage permissions. Users will be assigned predefined roles such as Administrator, Manager, or Employee. Each role will determine which actions or resources the user is allowed to access.

User Management
Administrators will be able to create new user accounts, update user information, assign roles, and revoke access when necessary. This functionality will help simulate how organizations manage employee access to internal systems.

Token-Based Authentication (JWT)
The system will use JSON Web Tokens (JWT) to authenticate and authorize API requests. Once users successfully log in, they will receive a token that allows them to securely access protected endpoints.

Audit Logging
The system will record important actions such as login attempts, role assignments, and permission changes. These logs will help track user activity and demonstrate how organizations monitor access for security and compliance purposes.

API Integration for Authentication
The IAM system will provide API endpoints that other applications could use to verify user identity and permissions. This simulates how real-world systems rely on centralized authentication services.

These features represent the core functionality of an Identity and Access Management system and will be demonstrated in the final project.

➢ Out-of-Scope Features (Explicitly Excluded)

The following features are intentionally excluded from the project scope to keep the system focused and achievable within the project timeline.

Full Web Interface or Frontend Application
The project will focus on backend services and API functionality. A full user interface or web dashboard is not included in the scope.

Single Sign-On (SSO) Integration with External Providers
Integration with external authentication providers such as Google, Microsoft, or social login services will not be implemented.

Multi-Factor Authentication (MFA)
Advanced authentication mechanisms such as SMS verification, email verification, or authenticator apps will not be included.

Enterprise-Level Security Infrastructure
Features such as advanced threat detection, intrusion monitoring, or enterprise security compliance frameworks are beyond the scope of this project.

Large-Scale Production Deployment
The system will be developed as a prototype for demonstration purposes and will not be deployed in a real production environment.

➢ Assumptions and Constraints

Several assumptions and constraints guide the development of this project.

First, the system assumes that users will authenticate using standard credentials such as username and password. Password security will be handled using hashing methods like bcrypt.

Second, the project will focus on backend API functionality using technologies such as Node.js with Express or Python with FastAPI. The system will store user data, roles, and logs in a database such as PostgreSQL or MongoDB.

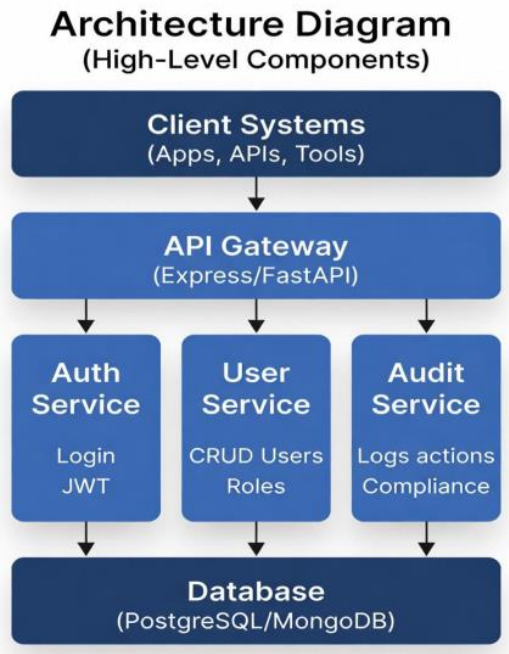
Another assumption is that administrators will manage users and permissions directly through API requests rather than a graphical user interface.

The project is also constrained by time and academic requirements. Therefore, the implementation will focus on demonstrating the core concepts of Identity and Access Management rather than building a full enterprise-grade security platform.

Finally, the system will be designed as a functional prototype that demonstrates authentication, authorization, and user management capabilities in a controlled environment.

Minimum Required Design

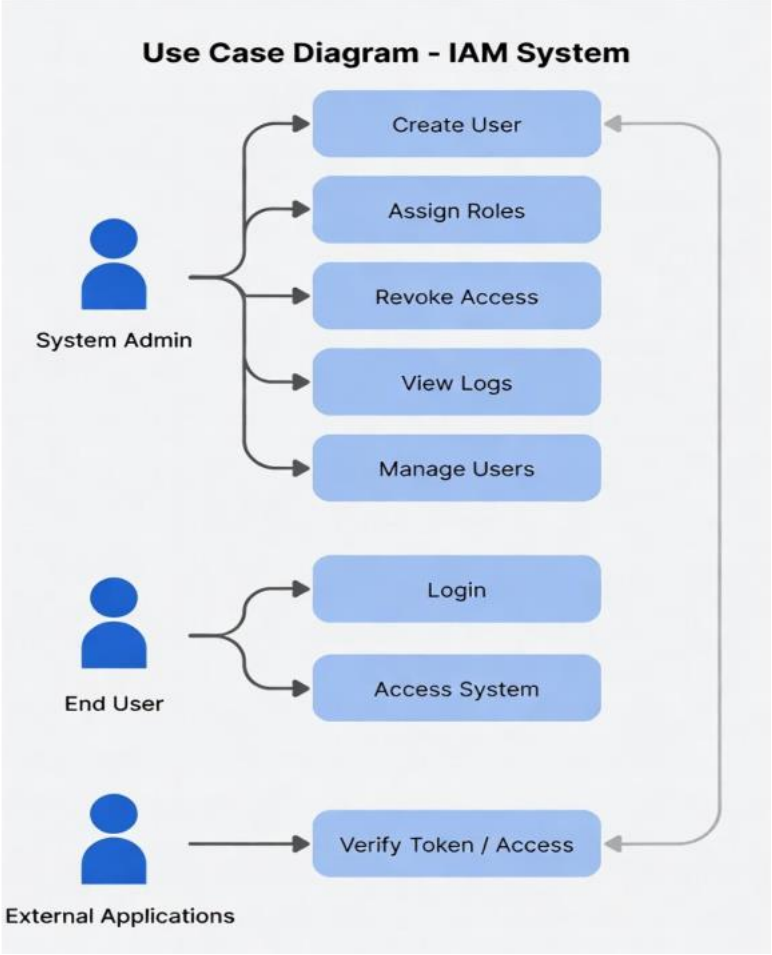
➢ System Architecture Diagram (High-Level Components)



➢ Use Case Diagram Use Case

1. User Login Actor:

End User



Preconditions:
- User account exists
- Valid credentials

Main Flow:
1. User sends login request
2. System validates input
3. Password checked (bcrypt)
4. User verified
5. JWT generated
6. Token returned
7. User uses token

Alternative Flow:
- Invalid credentials → error

Postconditions:
- User authenticated
- JWT issued

Use Case 2: Create User
Actor: System Administrator

Preconditions:
- Admin authenticated
- Admin has permissions

Main Flow:
1. Admin sends request
2. System validates data
3. Password hashed
4. User created
5. Role assigned
6. Action logged

Alternative Flow:
- Invalid data → rejected

Postconditions:
- User created

Use Case 3: Assign Role
Actor: System Administrator

Preconditions:
- User exists
- Admin authorized

Main Flow:
1. Admin selects user
2. Admin selects role
3. System updates role
4. Permissions updated
5. Action logged

Alternative Flow:
- Invalid role → error

Postconditions:
- Permissions updated

Use Case 4: Access Resource
Actor: End User / External System

Preconditions:
- Valid JWT token

Main Flow:
1. Request sent with token
2. Token verified
3. Role extracted
4. Permissions checked
5. Access granted
6. Action logged

Alternative Flow:
- Invalid token → rejected
- No permission → denied

Postconditions:
- Access granted or denied

Use Case 5: Revoke Access
Actor: System Administrator

Preconditions:
- User exists
- Admin authorized

Main Flow:
1. Admin selects user
2. Access revoked
3. System updates status
4. Tokens invalidated
5. Action logged

Alternative Flow:
- User not found → error

Postconditions:
- User access removed

Use Case 6: View Audit Logs
Actor: System Administrator

Preconditions:
- Admin authenticated

Main Flow:
1. Admin requests logs
2. System retrieves logs
3. Logs filtered
4. Logs returned

Alternative Flow:
- No logs available

Postconditions:
- Admin can review activity